SensorAct: Privacy Aware Scriptable Middleware for Monitoring and Control in Buildings

Pandarasamy Arjunan[1,⋆], Manaswi Saha[1,⋆], Haksoo Choi[2], Manoj Gulati[1], Amarjeet Singh[1], Pushpendra Singh[1], and Mani B. Srivastava[2]

[1] Indraprastha Institute of Information Technology, Delhi, India
{pandarasamya,manaswis,manojg,amarjeet,psingh}@iiitd.ac.in
[2] University of California Los Angeles, United States
{haksoo,mbs}@ucla.edu

Abstract. Buildings, with their different subsystems interacting with diverse occupants, constitute a complex cyber-physical-human infrastructure. Monitoring and controlling this complex ecosystem is essential both for efficient and optimized operations of building subsystems and for influencing the occupant behavior. A critical enabling technology in this case is a building middleware system that can fuse and analyze intentionally acquired or opportunistically available information from diverse embedded sensors, human feedback, and existing building subsystems, and enable rich inferences derived from this information. This paper presents *SensorAct* - a building middleware that addresses the core requirements of such systems: *privacy-awareness*, *scriptable-control*, *scalability across different buildings*, *support for heterogeneous platforms*, *extensibility to emerging technologies*, and *usability across diverse occupants*. We describe the detailed system architecture and design, and provide proof of concept through multiple third party applications built using *SensorAct* API and deployed in diverse settings across India and United States. *SensorAct* is released in open source for community use.

Keywords: privacy-aware, scriptable middleware, monitoring and control, buildings

1 Introduction

Buildings, both residential and commercial, subsume several subsystems such as Heating Ventilation and Air Conditioning (HVAC), security and fire, interacting with occupants with diverse behaviors. Understanding the complex cyber-physical-human interactions in the buildings require detailed monitoring which can then be used for optimized control for each of the subsystems. Of particular relevance is energy consumption across different subsystems. Buildings account for significant proportion of overall energy use in both developing (e.g. 47% of total energy in India [11]) and developed (e.g. 41% and 45% in US and UK respectively) countries. Modest improvements in building energy use can result in significant aggregate impact at national level.

⋆ The first two authors contributed equally to this work and are listed in alphabetical order

Building subsystems exhibit diverse facets across the globe, many of which are also informed by our own first hand experience in managing building systems across two countries - US and India. Occupant diversity, spread across socio-economic and cultural dimensions, further complicates the building-human interactions. Detailed understanding of different subsystems within buildings, together with their interactions with diverse occupants, is critical for developing solutions that optimize the overall operations of the building and can scale across diverse usage patterns.

Commercial buildings often employ Building Management Systems (BMS) for monitoring and control of its subsystems. While striving to achieve an optimal and energy efficient control, these BMSs are often very expensive and restrict the management to a central facility department. Although these BMS partially allow individual occupants to exercise their control on building operations, this control often lacks personalization e.g. for a thermostat in a shared space, the system only records the thermostat setting and not who has performed this setting. Participatory engagement of building occupants that involves not only gathering the required information from different *spaces* spread across the building but also associating them with specific building occupants will likely result in optimized usage and higher user comfort. Personalization of monitoring and control also requires integrating data from sources other than those typically supported by commercial BMS, such as tags and mobile phones. However, such personalization brings along important data privacy and control security concerns which should be appropriately addressed. Correspondingly, while BMS is for managing "buildings", *SensorAct* proposes privacy-driven and personalized management of "spaces" within and across buildings.

Together with supporting diverse usage patterns and personalization without compromising the data privacy and overall operational security, a middleware for building management should - (i) accommodate a rich ecosystem of existing and new monitoring and control systems, (ii) allow for easy development of higher level third party applications thus promoting innovation similar to the mobile phone segment, and (iii) be easily deployed by a non-technical user for small settings such as homes and campus dormitory rooms. Motivated by these requirements, we present *SensorAct* - a privacy-aware and scriptable middleware for detailed monitoring and control in buildings, while involving personalized and participatory engagement of the occupants. The key contributions of our work are:

- A working middleware system, released in open source, that can run on heterogeneous platforms, allowing participants to collect data from a variety of sensors and perform control actions thereof.
- *Fine-grained access control* allowing sharing of sensor data and actuation control with other participants.
- A scripting framework to enhance the system functionality with customized application logic.
- An evaluation of the system with respect to usability, extensibility and data access and control.

A preliminary design of *SensorAct* was presented earlier in 2012 Workshop on Embedded Systems for Energy Efficiency in Buildings (Buildsys) [3]. Significant

system development leading to multiple deployment studies and extensive system evaluation are all new inclusions for this paper.

2 Related Work

In this section, we explain about existing works related to monitoring and control with in buildings. We classify them into two categories (i) Commercial systems and (ii) Research systems, frameworks and architectures.

2.1 Commercial Systems

Several commercial building management systems (BMS)[3] and home automation[4] systems are currently available in the market for monitoring, controlling and automating various building subsystems and operations. Typically, they comprise of several isolated subsystems each performing a particular task such as fire alarms, security and access control and HVAC. While they include large number of sensing points spread across the building, data from these sensors is usually inaccessible to building occupants as these systems are normally controlled and managed by a central facility department. While many commercial buildings already have some form of BMS already in existence, *SensorAct* can be used to augment them to develop novel occupant-centric applications such as personalized control of workspaces. This is possible in *SensorAct* using Machine-to-Machine (M2M)[5] applications as these systems internally use several open standard protocols such as Bacnet/IP, Modbus and OLE for Process Control (OPC) for their monitoring and control operations. Further, higher costs typically associated with such BMSs, prohibit their usage across small deployments such as in residential homes. Home automation systems, try to fill in the gap providing comfort to the occupants, using local storage and several automation scripts. However, they provide limited support for fine-grained data and control sharing across multiple homes, supported extensively in *SensorAct*.

Several cloud based sensor data aggregation systems providing features to collect, store and visualize the real time data feeds have been proposed recently[6] for applications pertaining to Internet of Things. Most of these systems provide limited support for sharing data with other users and they follow the basic "all-or-nothing" model based only on the user identity. On the other hand, *SensorAct* supports selection of sharing model based on several context information, sensor values and across individual users and user groups. Though, a few services such as Sen.Se provide remote actuation support from Internet, they handle simple use cases wherein control is manual or is based on simple events e.g. whenever motion is detected, switch an appliance. *SensorAct* architecture allows for easy support of complex control mechanisms.

2.2 Research Systems, Frameworks and Architectures

Several research systems pertaining to connecting and sharing sensors at Internet scale, such as SenseWeb [12], SensorWeb [8], GSN [1], and WattDepot[4]

[3] `http://www.trane.com`,`http://www.johnsoncontrols.com`

[4] `http://micasaverde.com`, `http://www.homeseer.com`

[5] `http://mango.serotoninsoftware.com`

[6] `http://xively.com`, `http://www.nimbits.com`, `http://open.sen.se/`

have been developed and deployed in the recent past. However, these systems are limited primarily to sensory data aggregation, visualization and provide minimal sharing capabilities. Integration and management of diverse devices with a computing platform is a challenging task due to device interoperability issues. Several research software systems have been proposed in the literature that provides an abstraction over diverse devices and enables uniform interfaces to access them [13, 10]. HomeOS [10], provides a PC like abstraction over the networked devices as peripherals for both users and developers. HomeOS design broadly addresses the interoperability and usability issues of home automation systems with limited support for occupant centric functionalities such as personalized appliance control as supported by *SensorAct*. Sensor Andrew[15], a large scale campus wide deployment of sensing and actuation system, has similar goals as *SensorAct*. It focuses on large scale data aggregation from diverse sensors and event based control for building operations. However, the data and control sharing mechanism is at coarse-grained level, based only on user identity. A high-level architecture for smart building control system focusing on integration of different domains of building operations is presented in [6].

BuildingDepot[2] focuses on managing network of buildings by isolating the data, users and privileges management from each other. On similar lines, Building Operating System Services (BOSS) and Building Application Stack [9] enable writing portable and fault-tolerant applications on top of diverse physical resources present in buildings. However, both BuildingDepot and BOSS focus primarily on commercial buildings and subsystems (like HVAC) therein. Additionally, guard rules framework in *SensorAct* provides much more detailed control over data and actuation than supported by any of these systems.

For residential homes, VHome[17] is a cloud based energy management and control system that provides isolated application execution environment for data analysis. Though VHome shares several design goals with *SensorAct*, it provides an application runtime to execute Cloud Based Application, focusing only on energy data analytics. Further, access control mechanism in VHome supports only time and location based energy data sharing, whereas *SensorAct* supports fine-grained sharing. While the existing research systems focused on different aspects of monitoring and control of building premises, to the best of authors knowledge, they either do not or minimally address privacy aspects, participatory engagement of building occupants and scalability from residential homes to commercial buildings, which form the core of *SensorAct* design goals.

Our motivation to model and use VPDS as a privacy preserving system is derived from other systems such as Personal Data Vault [14], SensorSafe [7], and privacy preserving proxy servers for mobile platforms[5][16]. These systems allow users to own their data on individual servers while still providing the flexibility for sharing the data and control with other system users. However, as they are not specifically designed for building monitoring and control, there is limited or no support for guarding the actuation in such systems.

3 System Architecture and Design

Primary objective of *SensorAct* is to provide a middleware that allows for detailed monitoring and control of building subsystems, while involving personalized and participatory engagement of the occupants through controlled sharing. We now provide design goals and example use cases that motivate the architecture design of *SensorAct* and provide rationale for the design choices made.

3.1 Design Goals and Use Cases

Following are the design goals for a building middleware system, primarily established based upon more than one year of our extensive building deployment experience. Some of these design goals also overlap with those established in [17] for energy monitoring in small homes.

- **Personalized management and control:** of different *spaces* inside the buildings
- **Fine-grained access control:** allowing building occupants to exercise complex data and control sharing mechanisms
- **Rich support for automation:** allowing for execution of complex (one-shot or persistent) control actions supporting sensor data processing, data fusion, actuator control and notifications.
- **Scalability:** from monitoring small homes to large commercial buildings, supporting diverse occupants and their interactions with different building subsystems.
- **Heterogeneity:** for support on diverse platforms from x86 to ARM ISA
- **Extensibility:** through programming abstractions facilitating seamless integration with existing building systems and development of third party applications
- **Usability:** such that the system can be easily setup, configured and maintained by a non-technical user.

Correspondingly, we define following example use cases for a middleware that translate several of the above design goals into real world applications, many of which are based upon our experience:

Use case I: A small homeowner wants to set up sensing and control within their homes. The system should be easy to setup on a small and low-cost single board computer (such as Raspberry Pi).

Use case II: Facility manager in a large commercial building wants to share the sensing and control with the building occupants such that occupants can decide who can see the data or control the actuators that belong to their rooms.

Use case III: Systems that enable users to share data and control, while ensuring privacy and security, beyond occupants (e.g. researchers or utility companies) will significantly contribute to the advances in building management systems.

3.2 System Architecture

We now discuss the architectural details of *SensorAct* that address the design goals and ensure the applicability across the three use cases discussed above.

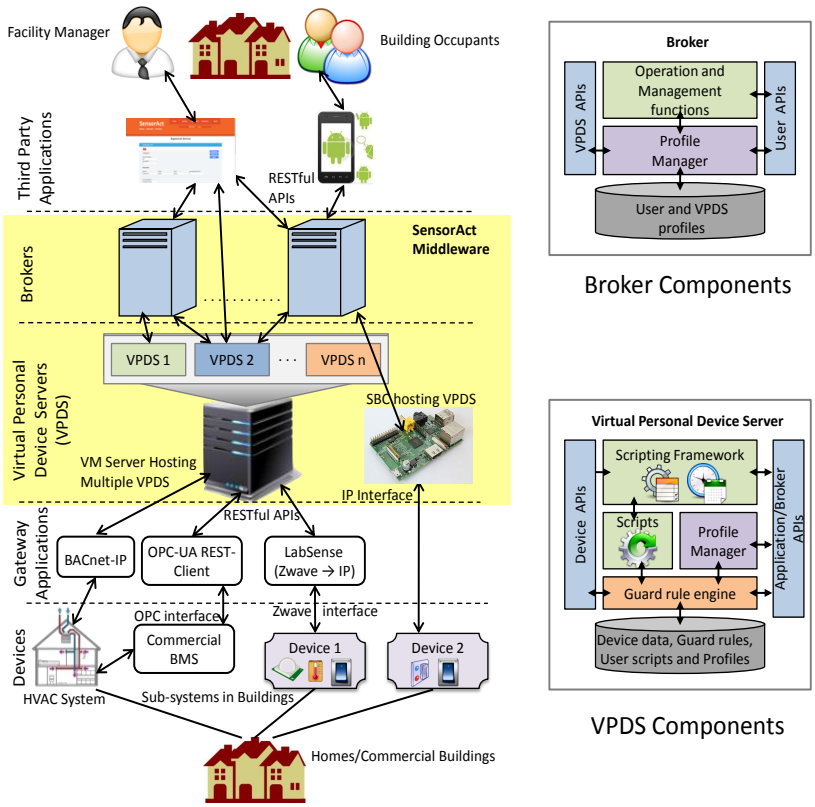


Fig. 1: Tiered architecture of *SensorAct* together with details of VPDS and Broker components

SensorAct middleware adopts a *tiered architecture* connecting the building subsystems with building occupants as shown in Figure 1. The main components of *SensorAct* are Virtual Personal Device Server (VPDS) and Broker, interacting with devices that monitor and control different building spaces at the lower layer and third-party applications at the higher layer using its RESTful APIs.

Devices: A device in *SensorAct* terminology consists of zero or more sensors (monitoring the ambient environment or the state of a building subsystem) and actuators (that allow for changing the state of a building subsystem). Devices are assumed to either communicate directly using the RESTful API (e.g. Device 2 in Figure 1 or through a gateway that can bridge the sensor interface to *SensorAct*(e.g. LabSense, discussed in Section 7.3, is used to bridge Device 1 supporting zwave with *SensorAct*).

A device profile consists of a set of attributes such as its name, IP address, a collection of sensor and/or actuator profiles, number of channels, data types and units, location and placement. Device profile is represented using hierarchical model to name and identify devices, sensors, actuators, channels, and their readings uniquely e.g. $building : floor : room : device : id : [sensor|actuator] :$

id : *channel* : *data*. A data upload or data query can consist of a set of these attributes. VPDS Owner (VO) can manage multiple devices owned by him through the *profile manager* facilitating creation of device profiles. Each of these devices is manually configured with an upload and actuation key, generated at VPDS, to authorize the devices for interaction with VPDS.

Virtual Personal Device Server (VPDS): is the primary core component of *SensorAct* middleware. As a package, VPDS contains a database, a *Scripting framework*, a *Guard rule engine*, a profile manager and APIs for devices, applications, and brokers to interact with the VPDS, as shown in Figure 1. To ensure scalability, a single physical server may host multiple VPDS, however virtualization is used to ensure the isolation of VPDS and privacy of the data within it. A single user owns a particular VPDS and is termed as VPDS Owner (VO) in *SensorAct*. VO may allow other users to have controlled access to the devices registered with the VPDS. These devices are assumed to monitor and control spaces inside the buildings occupied by VO. Each VPDS has an associated owners key which is automatically generated and is used to register a VPDS with a broker. All the communication between VPDS and VO is authenticated using the owners key.

A schema-less, key-value based database is used to store the sensor data from devices, allowing for efficient storage and querying of unstructured time-series data. Access to the database is guarded via the *guard rule engine* (see Section 4) in the VPDS. Privacy of the *SensorAct* user is ensured using the guard rule engine together with the concept of local data storage inside the VPDS (supported through virtualization on the server or local installation inside a home) that acts as a vault for the data coming from the devices owned by VO. Lightweight scripts are used to act on configured triggers to support complex automation tasks (see Section 5).

The VPDS abstraction not only permits flexible provisioning of hardware resources but also allows diverse deployment scenarios ranging from a VPDS hosted on a local machine for high-rate, low dependence on internet connectivity, high-security and low-latency sensing and control to a VPDS running on the cloud-based hosting services such as Amazon EC2 providing low-maintenance. Multiple such VPDSs instances can be coordinated through the higher tier of Brokers. VPDS-Broker architecture further enables global access through distributed registry of users.

Broker: In *SensorAct*, a trusted broker contains a registry of users, a registry of VPDSs, and acts as a mediator to help clients establish a connection with VPDSs. A VO needs to register her VPDS on one of the broker to share data and control with other users registered on the corresponding broker. For data communication to scale across multiple VPDSs and users managed by a single broker, direct communication over a secured is provided between users and VPDSs (see Section 4 for details).

Applications: Programming abstractions in *SensorAct*, using RESTful APIs, allow easy development of third party applications that are used to provide a user with access to sensor data and actuator control. A *SensorAct* user may

have two roles: i) Owner of his/her own devices and correspondingly associated VPDS ii) User with data and control access as per the privileges assigned by the owner other devices. A single user could be the owner of his/her own VPDS and user for some other VPDS. As an owner, a user may grant controlled access to other users thus letting them access sensor data from her devices or to even control them in a carefully constrained fashion. Third party applications may provide users with more convenient interface to the underlying functionality of VPDS and broker. We discuss two such third party applications in Section 6.

4 Guard Rule Engine: Sharing Access and Control

The *Guard Rule Engine* is one of the core features of VPDS and acts as the central controller for the data and control access. It supports *privacy-aware sharing* i.e. "sharing of data and control governed by the corresponding owner". All access requests, for actuators in the devices or for data in the database, are authenticated by the Guard Rule Engine. Tight control on the access is maintained through user-defined *guard rules*. Guard rules are the policies defined by the VPDS owner for restricting access to the data and control of the devices configured for her VPDS. Guard Rule Engine enables *fine grained access control* by allowing the owner to define rules based on user, group, time, location, value of sensors, actuators and context inferences drawn from raw sensor data such as occupancy. Although our current prototype system includes preliminary implementation of Guard Rule Engine, in the following sections we discuss details of the complete guard rule feature design, planned for implementation in the near future.

Every device registered with the VPDS has an associated set of guard rules, created by the VPDS Owner (VO), for facilitating external (or shared) access. A guard rule is defined, along with its name and description, using the following attributes:

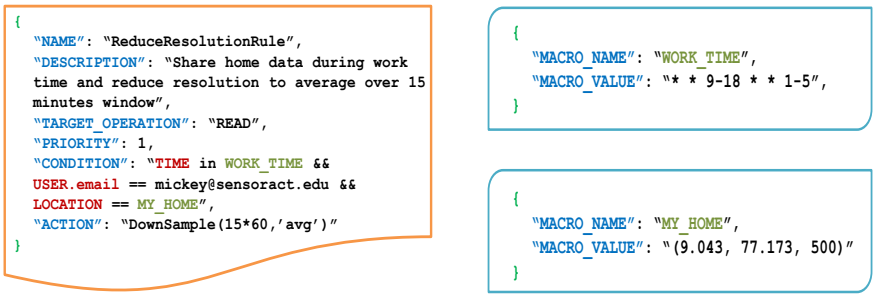
Target Operation: Type of access to which this rule is applied. It can be either *read* or *write* operation.

Condition: Boolean expression defining the condition when this rule is activated, e.g., who to give access and during what time.

Priority: Numeric value for assigning priority level for the rule which is used for conflict resolution during the rule enforcement.

Action: Action to be performed based on the condition of the rule, e.g., allow, deny, or applying data obfuscation functions.

An example of a guard rule is shown in Figure 2a. The guard rule enforces the policy to share home data only during work hours and reduce the data resolution to 15 minutes. The DownSample function is an example of a built-in function which is explained later. The guard rule itself does not contain reference to specific sensors or actuators but it is associated to them later on by the users as in this case with the AC actuator. This *lazy-association* allows users to reuse same rules for multiple devices or groups of devices.



(a) Guard rule with macros as condition (b) Macro definition

Fig. 2: Use of Macros in guard rules

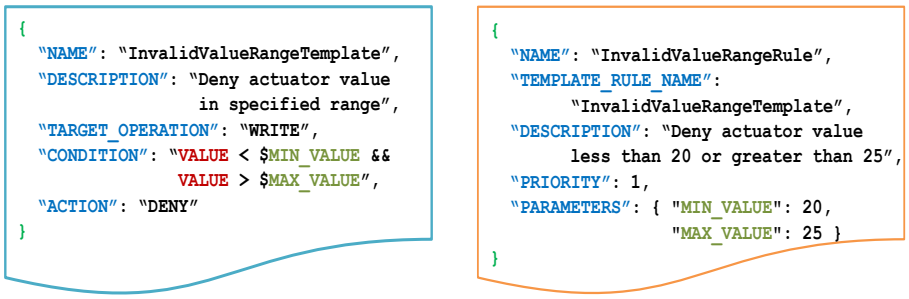
Privilege Sharing: Besides sharing data and control access, building applications may sometime also require an ability to give permissions for administrative tasks to individual or group of building occupants. Considering our *Use Case II*, the facility manager (VO) may allow occupants to create rules or actuation scripts for the devices in the spaces occupied by them. Specifically, such administrative tasks in *SensorAct* include creation, deletion, and modification of device profiles, guard rules, and automation scripts, facilitated by the Guard Rule Engine and controlled by the VO. This feature is also used in our user study, discussed in detail in Section 7.2.

Advanced Guard Rules: Guard Rules are envisioned to be reused reducing the repeated effort required for creating similar rules multiple times. To facilitate this reuse, we employ the concept of macros, templates, and built-in functions. These two constructs are used to create custom conditions corresponding to the real world usage, from a generic definition that can be defined once by some experts or third party applications. Using the examples shown in Figure 2 and Figure 3, we demonstrate the effectiveness of using macros and templates:

Macros: are used to name an input sequence which is usually a pattern such as a time period or location coordinates. In Figure 2b, $WORK\_TIME$ and $HOME$ are examples of macros. Defined macros can be applied to any guard rule in the system and hence enable flexibility in creation of specific guard rules.

Templates: are used to create parameterized guard rules. This is useful when guard rules have a structure that can be associated with multiple devices with a minor change in some of the parameter values. Figure 3 gives an illustration of a guard rule template that defines a policy for invalid range of values for actuators. This template can be applied to multiple actuators by instantiating it with device-specific range values. For example, for AC actuators, the invalid range can be less than 15 and more than 25; for electric fans the invalid range can be less than 1 and greater than 5.

Built-in Functions: are used to obfuscate the data while preserving the captured events. This can be particularly useful to preserve privacy of a user while extracting the desired inference from the data. An example is the Downsample function, shown in Figure 2b, it averages out the home data over a 15 minute



(a) Template Definition (b) Template Instantiation

Fig. 3: Templates in guard rules

window. This hides the specific events while sharing the aggregated energy usage information.

The distributed architecture of *SensorAct* allows for creating separate VPDS for individual spaces within and across buildings. Brokers enable sharing and access of data and control across multiple VPDS. Please refer to the earlier draft of this paper [3] for details about the sharing process.

5 Scripting Framework: Providing Simplified Automation

Existing building management systems typically provide a stand-alone package of fixed, pre-configured and limited support for control and automation. Their archaic nature limits integration of different subsystems (e.g. information from access control system to appropriately control HVAC for workspaces), addition of new sensors and development of novel occupant centric building control applications. Legacy programming models and application specific jargons further make it difficult to extend the system by someone who is not a domain expert.

The *Scripting Framework* in *SensorAct* abstracts out the underlying complexities and provides a set of simple primitive read and write APIs, in a sandbox application execution environment. This enables building occupants to inject their custom application logics, termed as *tasklets*, into the middleware to perform sophisticated management and control operations while the system is running. These read and write primitives can then be used to develop complex (one-shot or persistent) control actions. Figure 4 illustrates the workflow of the proposed scripting framework. It enables "on-the-fly" application development environment for buildings, for several purposes including, but not limited to 1) inference of high-level rich occupant-specific context information, such as occupancy and usage patterns, by fusing several raw sensor and actuator values; 2) automation of the routine building control activities performed by the occupants in their day-to-day life e.g. pre-heating or pre-cooling the workspaces in advance; 3) custom creation of *coordinated appliances* based upon detected building events e.g. switching off a meeting room may result in switching a group of devices together or in a particular sequence; and 4) development of an automated alerting and notification system for monitoring and management of building premises e.g. alert the facility management team in case of any failure in HVAC.

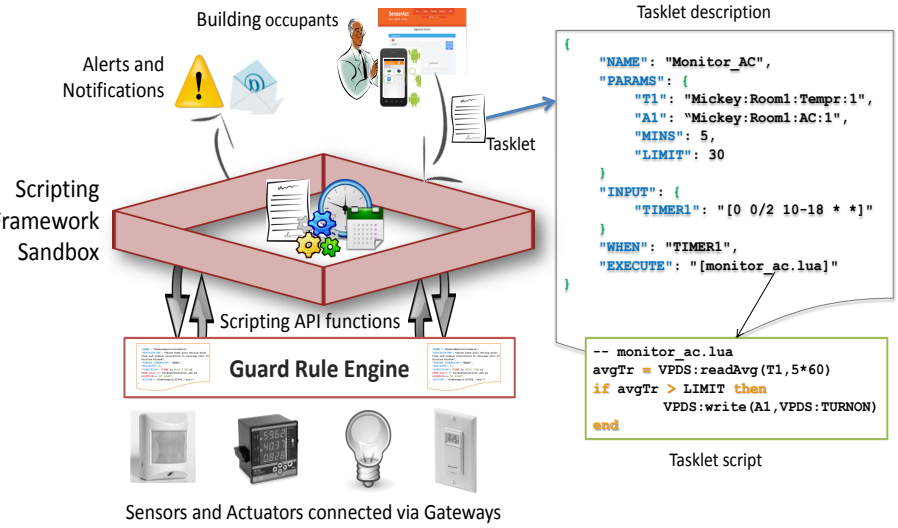


Fig. 4: Scripting framework - workflow

5.1 Tasklet Workflow

A *tasklet* in the scripting framework is a piece of light-weight non-blocking code that performs a particular set of operations. As shown in Figure 4, it consists of *tasklet description* and *tasklet script*. The tasklet description follows a simple programming model which consists of following primitives to specify the context of the tasklet:

Param: contains a key-value pair of various runtime parameters required by the tasklet. It may consists of numeric and string constants and notification specific parameters such as email address.

Input: contains a key-value pair of name and corresponding identifiers for sensors, actuators and timers (cron expression format) that act as triggers for the action specified in the tasklet.

When: specifies a condition that triggers the execution of the script upon valid satisfaction. It consists of a combination of min-terms (logical AND) and max-terms (logical OR) of names specified in the *Input* primitive.

Execute: contains the tasklet script to perform when the condition in the "When" clause holds true.

A *tasklet scheduler* receives tasklet execution requests and is responsible for scheduling, management and control of the tasklet through its life-cycle. Once a tasklet is submitted for execution, the tasklet scheduler first classifies and schedules the tasklet, based upon the identifiers used in the *When* primitive, as per the following categories:

One-shot: a nil triggering condition will yield to one-time and immediate execution of the action script. e.g. querying the current status of a sensor or instantaneous switching of an actuator.

```
table  VPDS:Read   (string DeviceId [, number seconds=LATEST])
number VPDS:ReadAvg(string DeviceId [, number seconds=LATEST])
table  VPDS:Read   (string DeviceId, number fromTime, number toTime)
number VPDS:ReadAvg(string DeviceId, number fromTime, number toTime)
boolean VPDS:Write(string DeviceId, number status=ON|OFF|value)
boolean VPDS:Notify(string userId, string message)
```

Table 1: Primitive scripting framework API functions

Periodic: a timer based triggering condition will yield to executing the action script whenever timer elapses to perform one-time or periodic operations. e.g. switch on my office Air-conditioner at 9AM on weekdays or email me the electricity usage summary of my office every day at 8PM.

Event driven: a sensor or actuator based condition will execute the action script whenever a change in the value of the sensor or the actuator is observed. e.g. switch off lights when window is opened.

Tasklet manager provides an isolated execution environment for each tasklet and restricts any interaction among them for sensitive building control applications. By default, a tasklet inherits the privileges of the user who submitted it. All the read and write operations on sensors and actuators invoked from a tasklet goes through the guard rule engine. Correspondingly, the tasklet can only perform the operations allowed for the user invoking the tasklet, thus protecting against unauthorized action.Figure 4 shows an example tasklet that *Read past 5 minutes average temperature of Room1 and turn ON the Air-Conditioner if the average temperature is above 30°C* with a triggering condition *Repeat this every 2 minutes from 10AM to 6PM only on week days.*

5.2 Tasklet API Functions

The scripting framework in *SensorAct* provides a set of primitive API functions, as shown in Table 1, in order to perform various runtime operations. Third party applications can use these APIs, to provide building occupants an opportunity to develop and schedule custom building control applications. While the experienced occupants can write complex tasklet scripts on their own, novice users can use an interactive web interface to design them easily. We plan to extend this API list with additional functions supporting comprehensive notifications such as messaging (SMS), voice call and interaction with social networks.

6 Implementation

We implemented all the VPDS and Broker components and APIs and the three sample applications (see Section 7.3) using open source softwares and libraries. We used Java based Play framework[7] to implement VPDS, broker, and the sample web application. Implementation in Java ensured easy portability for heterogeneous platforms supporting x86 and ARM ISA. Play framework provides inherent support for several libraries such as JSON, validation, jobs and testing tools which help in quick implementation. Further, Play web applications can be deployed directly to cloud platforms, such as Google App Engine, with minimal

[7] `http://www.playframework.org`

Component	VPDS APIs
User	/user/{register|list}
Key	/key/{generate|delete|list|enable|disable}
Device	/device/{add|delete|get|list|search|share}
	/device/template/{add|delete|get|list}
Guardrule	/guardrule/{add|delete|get|list}
	/guardrule/association{add|delete|get|list}
Tasklet	/tasklet/{add|delete|get|list}
	/tasklet/{execute|cancel|status}
Data	/data/{upload/wavesegment|query}
Share	/data/{upload/wavesegment|query}

Component	Broker APIs
User	/user/{register|login|list}
VPDS	/vpds/{register|get|list}
Device	/device/{search|share}
	/device/{user|owner}/shared

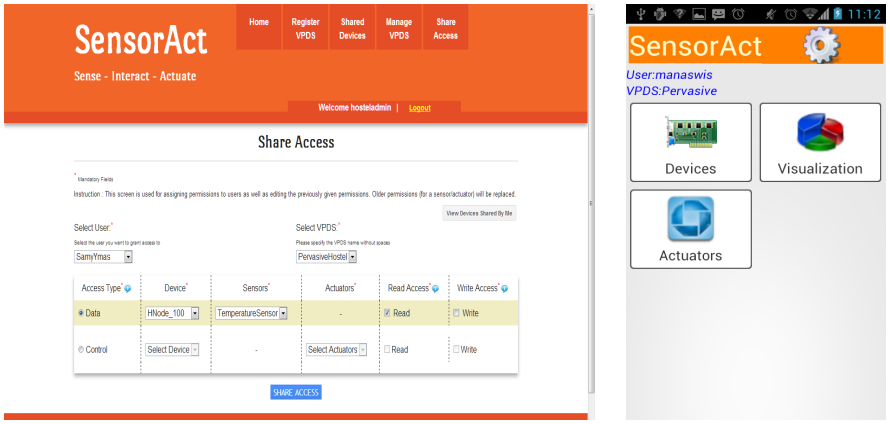
Table 2: Supported SensorAct APIs for different system components

modification. Both VPDS and Broker use MongoDB, a schema-less, document based, NoSQL, and highly scalable database to store sensory data, profiles, and configuration information. MongoDB is suitable for writing intensive database operations, thus satisfying the prime requirement to aggregate high frequency sensory data.

SensorAct exposes a rich set of RESTful developer APIs for most of its functionalities. Such open APIs enable easy integration with other systems, such as Xively, Nimbits and MangoM2M, and allow developers to write custom third party (stand-alone, browser based, or mobile based) applications, e.g. for visualization, scripting, access control, and sharing. Table 2 lists the APIs currently implemented for various components in VPDS and Broker. We have also documented the functionality, request and response format and error codes for each API elaborately. Each API request is authenticated using a *secretkey* which is generated and associated with each user when they sign up. We use JSON, a light-weight and flexible data interchange format, to receive and send API request and response data.

In the current implementation, *SensorAct* natively supports only push based sensors i.e. sensors pushing the data to VPDS using its RESTful interface, however, pull based sensors can be easily incorporated using third party gateway applications. Currently, we use two such gateway applications - LabSense[8] gateway to pull data from Z-Wave based sensors and a pymodbus based python application to pull data from modbus enabled smart meter. The current implementation of guard rule engine supports user email id, time, and value based access and control sharing for sensors and actuators. We are working on extending the guard rule engine to support other features such as sensory data obfuscation, macros and down sampling as discussed in Section 4.

[8] http://github.com/jtsao22/LabSense



(a) Browser application (b) Mobile application

Fig. 5: Sample browser and mobile application user interface

We use Java based Quartz Scheduler[9] to implement our scripting framework. It is a high-performance, industrial strength job scheduler which scales well to manage large number of tasklets. We use Lua to write the tasklet scripts. Lua is a light-weight, small footprint, compact and easy-to-learn scripting language which has also been widely used to script the home automation systems, Linux kernel development and packet filters. Java to Lua integration was done using jnlua binding engine which is invoked via Java Scripting API framework from the tasklet manager. Several higher level scripting API functions, listed in Table 1, are exported into Lua interpreter's execution context while triggering tasklets so that they can be directly called from the user written tasklet scripts. We further plan to strip down Lua source code and integrate some static code analysis techniques to identify and block potentially harmful codes before executing the tasklet scripts. The scripting framework can easily be extended to support additional scripting languages such as JavaScript.

We have also implemented a stand-alone web application (illustrated in Figure 5a) that interacts with a broker (configurable) and several VPDSs using the *SensorAct* APIs listed in Table 2. Both VPDS owner and building occupants can use this interface to manage devices, guard rules and tasklets based upon the granted privileges. An integrated visualization application enables users to plot their sensory data. We used Play framework and several Web 2.0 technologies to develop this web application. In order to simplify the installation procedure, we packaged all software modules into a Virtual Machine image and created a detailed installation instruction manual to install and configure the *SensorAct* system on a computer.

7 Evaluation

We evaluate *SensorAct* on the basis of: usability, extensibility and system performance. Usability is demonstrated through multiple cross-border deployments

[9] `http://quartz-scheduler.org`

involving diverse sensors and different sharing requirements. Extensibility is validated through multiple external applications built using programming abstractions provide by *SensorAct*. System performance is evaluated across heterogeneous computing devices hosting *SensorAct* and using performance of data upload and data query that are the two common operations made in *SensorAct*.

7.1 Deployment Experiences

We deployed *SensorAct* in a research wing at IIIT-Delhi, India and a research lab in UCLA, USA. In addition to the the two big deployments, *SensorAct* is also hosted on RaspberryPi system for collecting ambient information (motion, temperature and light) from a home in Los Angeles and smart meter data from a home in Delhi, demonstrating the wide applicability of *SensorAct* across different deployment scales. Our deployments resemble the use cases as explained earlier. Each of the institution and home deployments hosted a separate VPDS. All the VPDSs were registered to a common broker, hosted at IIIT Delhi, for sharing the data with researchers across the two organizations. This sharing can be extended to other researchers to facilitate research collaborations.

Research Wing at IIIT-Delhi: The first deployment, at IIIT-Delhi, involved detecting occupancy from low cost sensors connected across 14 WiFi based sensor nodes, built by extending the Flyport[10] hardware. These sensor nodes were spread across 9 rooms of which 6 were individual faculty offices and 3 were shared spaces constituting PhD and Masters labs. Each sensor node was monitoring temperature, motion and light data every second and was communicating it to *SensorAct* every 10 seconds over 802.11b network. Corresponding VPDS, hosted inside a virtual machine on a server (Dell PowerEdge Windows Server) is managed by one of the faculty member while other occupants are registered as users on the broker (also hosted on one of the servers at IIIT Delhi) to which the VPDS was registered. *SensorAct* allowed the faculty member to provide other wing occupants with privileges to share the data from the devices that belong to the spaces occupied by them, similar to Use Case II. Over more than 2 months (currently ongoing) of deployment, *SensorAct* was able to aggregate all the received sensor data accurately, except for the network errors. Android application (discussed in Section 7.3) was given to wing occupants to allow them to visualize the data shared with them. The deployment is currently used by other researchers for multiple user studies. Wing occupants now decide which study they want to be part of and accordingly data from only those devices is shared with the researchers.

Research Lab at UCLA: The second deployment, at UCLA, involved occupancy detection using existing sensory information and verifying it with ground truth data collected using externally deployed sensors. VPDS instance, in this case, is hosted on a Linux Virtual Machine (VMware vSphere ESXi hypervisor) on a Intel Xeon processor based DELL PowerEdge R420 server. Data in this deployment is collected from - 1) high frequency multi channel Rariton, Eaton, and Veris power meters measuring several electrical parameters of the lab servers

[10] `http://www.openpicus.com/`

and power outlets and communicating over SNMP and 2) Z-Wave based Aeon Labs door sensor and HomeSeer multi-sensor, measuring motion, temperature and light, interfaced with Mi Casa Verde Vera. We use Labsense as a gateway to pull data from all of these sensors. The VPDS instance has been running for the past one month and is registered with the broker hosted at IIIT-Delhi. Access for sensor data and creating tasklets is shared with one of the PhD student at IIIT-Delhi who created a computed sensor for presence detection (discussed in Section 6) and is performing occupancy based experiments.

7.2 Case Study: Dormitory Deployment at IIIT-Delhi

At IIIT-Delhi, 21 student groups (each with 2 students) were engaged to deploy the Flyport based nodes in dormitory rooms, collecting motion, temperature, and window status information every second. Most of the students in the study did not stay in the dorms so were given the flexibility to deploy these systems in any dorm room occupied by their friends. A total of 17 groups deployed sensors in their friends room and 4 groups deployed sensors in their own room. Each of the student group was asked to create a local VPDS on their own computer to verify the correctness of the deployment. The students reported local VPDS deployment ranging from 2 days to 15 days in the post-study survey.

Thereafter, one of the faculty coordinator hosted a central VPDS on one of the servers and registered it with the broker at IIIT-Delhi. Students were then asked to register themselves and the room occupants (for the case when they are deploying in someone elses room) on the broker. Faculty coordinator first shared privileges with the engaged student groups and allowed them to create devices in his VPDS though they were not allowed to see data from the devices added by them. This privilege was revoked after 2 days and the added devices were shared with the corresponding room occupants to let them decide who they want to share the data with. While, the study mandated each student to collect only 2 days worth of data from their individual deployment, a total of approx. 100 days worth of data, together from all the groups was reported by the students.

Observations: *SensorAct* architecture helped in addressing the complex deployment scenario whereby the occupants were different from those deploying the sensors and were given the control to decide if the deployers should be able to collect and view the data from their rooms, similar to use case I. Further, *SensorAct* easily allowed giving temporary privileges to the students to add a device in the central VPDS and revoke them at a later stage.

A survey was done at the end to get the feedback from these students on different aspects of *SensorAct* system. Overall 17 student groups responded to the survey at the end of the study. 16 student groups had no prior experience with uploading data to a server or cloud system such as *SensorAct*. More than 80% of the responses mentioned that *SensorAct* installation and configuration of *SensorAct* on their individual laptops, with varying OS, was easy. Approximately 45% of respondents gave their preference for local hosting of *SensorAct* VPDS instead of cloud, due to privacy reasons, if they were to deploy *SensorAct* for monitoring and control in their homes. Students used *SensorAct* for basic data collection and visualization while the occupants used sharing to provide access

of their devices to their friends. 64% of them found *SensorAct* to be a good and usable system. More than 90% of responses rated *SensorAct* documentation to be detailed enough with 73% of them asserting that, with the current level of documentation, a new person can setup *SensorAct* system without any help. More than 90% of the responses were positive about the usability of the browser application that was used as a front end for the study.

7.3 Third Party Applications Using *SensorAct*

We now illustrate applications developed by us, using the programming abstractions in *SensorAct*, to extend the system as per the deployment requirements.

LabSense: project is extended to enhance *SensorAct* with the capability to pull data pulled from SNMP based Raritan Dominion PX8 power distribution unit, Modbus based Veris Power Meter and Zwave based multisensor and door sensor, and push it to the VPDS deployed at UCLA.

Time and Presence Based Actuation: Browser based UI application is extended with a graphical UI, providing an intuitive interface over *SensorAct*, that allows users to remotely actuate their owned devices based on time and presence. The user interface used tasklets at the backend to allow users to switch their appliance "now", "once" at a specified time or "periodically" at a specified time. Further, users are provided with an option to specify the number of seconds for which to monitor the motion sensor in their occupied space and accordingly switch the appliance. Guard rules are used to allow users to actuate only the devices in their occupied space. This system is currently under deployment in a commercial building for lighting control.

Android visualization application: A simple Android based application (illustrated in Figure 5b) is developed whereby users can specify their Broker credentials and correspondingly can visualize the data from the devices owned or shared by them. This was used during the user study for students to remotely monitor the data from the dormitory deployment. This application is enhanced with data collection from sensors within the mobile for a user study in the research wing at IIIT-Delhi.

7.4 System Performance

In this section, we evaluate *SensorAct* based on its performance on both a single server and across multiple devices hosting *SensorAct* .

Scalability: We conducted two experiments to test the performance of *SensorAct* with increasing traffic load. We ran both the experiments on a server hosting *SensorAct* inside a virtual machine with 4 processor cores and 8GB RAM allocated to it.

To test for the performance with upload data requests, we considered a total of 10,000 requests and performed concurrent upload requests ranging from 100 to 1000 at a time. By concurrent request, we mean multiple requests hitting the *SensorAct* server at the same time. The upload packet within each request was 340 bytes. Using *ab*, Apaches benchmarking tool, we determined that *SensorAct* is able to handle 183 upload requests/second on an average even if larger number of requests are made concurrently. Remaining requests are buffered inside the

play framework. The average time to serve a single upload request was approx. 5ms. This is reasonable for most cases where a single VPDS instance caters to a single building with low frequency sensors deployed. Results are summarized in Table 3.

Number of concurrent upload requests	**Time to serve (ms)**	**Mean Response Time (ms)**
100	5.3	535
500	5.4	2643
1000	5.5	5268

Table 3: Performance evaluation of *SensorAct* for data upload

To test for the query response time, we varied the number of concurrent queries from 5 to 20 together with varying size of the query from 1000 records to 20,000 records for each set of concurrent requests. As shown in Table 4, we observe that the mean response time increases with the query size. The current implementation, does not involve any optimizations related to query processing and each query is a simple data fetch request.

No of Records fetched	**No of concurrent requests**			
	5	10	15	20
1000	0.2	0.3	0.7	0.9
5000	1.1	2.4	3.4	4.5
10000	2.8	5.6	8.2	10.1
20000	4.9	9.8	13.5	20.1

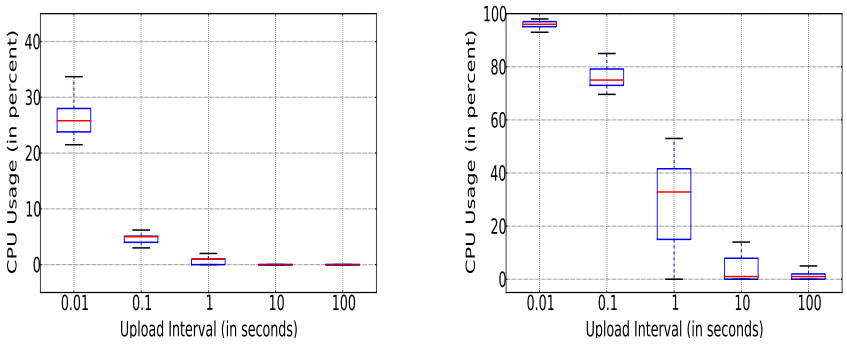
Table 4: Performance evaluation of *SensorAct* for query response

Various use cases and actual deployment experience illustrates that (frequent) data upload requests are more critical than (infrequent) data queries. Performance of *SensorAct* exhibited for both the data upload and query processing serves the purpose of a typical deployment scenario. For instance, the time spent to serve one data query request was 3 seconds for the largest query made. According to the deployment setting at IIIT-Delhi where the sensors send data every 10 seconds, it is equivalent to *2.*5 days worth of data from a single sensor. The smallest query made, which is 6 hours worth of data, took 50 ms on an average. Therefore, even with the current unoptimized implementation, *SensorAct* is able to support real world deployments with reasonable efficiency.

Heterogeneity: We hosted *SensorAct* on multifarious devices, shown in Table 5, ranging from high end servers to low end single board computers without any modification to *SensorAct*.

Devices	**Processor**	**RAM**	**ISA**	**OS**
Dell PowerEdge Server*	Intel Xeon 2.5 GHz	64GB	x86	Windows
Standard Desktop PC*	Intel Core 2.4 GHz	2 GB	x86	Linux
Mini ITX	Intel Dual Atom 1.86GHz	1 GB	x86	Windows
Raspberry Pi	Broadcom BCM2835 700 Mhz	256 MB	ARM	Linux

Table 5: Diverse systems hosting *SensorAct* (*using virtualization software)



(a) For Server (b) For Raspberry Pi

Fig. 6: CPU usage with varying upload interval

We conducted an experiment to test the performance of the three devices hosting *SensorAct* , viz. a high-end server, a desktop PC and Raspberry Pi. We demonstrate the performance of the device, in terms of CPU and memory usage, as upload interval for sensors is increased from 0.01s to 100s. We keep sampling frequency constant at 100Hz. We chose the upload intervals based on general deployment scenarios where the sensors might range from high frequency sensors (e.g. smart meters) to low frequency sensors (e.g. motion, light sensors etc.) and often will have small local storage to support for high rate sensing and lower upload rate. The server was hosting *SensorAct* in a virtual machine with 8GB memory and 4 cores allocated to it. *SensorAct* on the desktop was hosted in a virtual machine with 1 GB memory and 1 processor core assigned to it. On the Raspberry Pi, due to both memory and processing constraints, it was natively installed in Linux.

Due to limited space we only provide illustrations for performance of server and Raspberry Pi. As expected, from Figure 6 we observe that the CPU usage is the highest in all three devices when the upload interval was 10ms which translates to high upload frequency. As we increase the upload interval, the CPU usage drastically reduces for the server (from 25% to 0.25%) and desktop (31% to 0.36%), thus making them fit for moderate frequency sensors. In contrast, for Raspberry Pi, the mean CPU usage reduces gradually from 86% to 10%. Due to its limited processing capacity and memory, Raspberry Pi is suitable for small deployments such as in homes. In all cases, the memory usage was seen to be constant: for server, it was 16.2%, for desktop 12.1% and Raspberry Pi 47.6%.

8 Conclusions and Future Work

In this paper, we present *SensorAct* - a privacy-aware and scriptable middleware for building management. We discuss several design goals and use cases for such a middleware, informed by our first hand experience in monitoring building ecosystem. *SensorAct* is architectured to achieve several of these design goals. Of particular importance are the Guard Rule Engine ensuring fine-grained privacy aware control and Scripting Engine providing users with a simple framework to create complex automation tasks. Validation of the developed system was done

using multiple deployments, from residential homes to commercial buildings, spread across Delhi, India and Los Angeles, USA. We are currently working on efficient implementation of the Guard Rule Engine, optimized query processing, and enhancing *SensorAct* to support inference and modeling from the collected data.

References

1. K. Aberer, M. Hauswirth, and A. Salehi. Global sensor networks. *EPFL, Lausanne, Tech. Rep*, 2006.
2. Y. Agarwal, R. Gupta, D. Komaki, and T. Weng. Buildingdepot: An extensible and distributed architecture for building data storage, access and sharing. In *Proc. of BuildSys*, pages 64–71. ACM, 2012.
3. P. Arjunan, N. Batra, H. Choi, A. Singh, P. Singh, and M. B. Srivastava. Sensoract: a privacy and security aware federated middleware for building management. In *Proc. of BuildSys*, pages 80–87. ACM, 2012.
4. R. S. Brewer and P. M. Johnson. Wattdepot: An open source software ecosystem for enterprise-scale energy data collection, storage, analysis, and visualization. In *Proc. of SmartGridComm*, pages 91–95. IEEE, 2010.
5. R. Cáceres, L. Cox, H. Lim, A. Shakimov, and A. Varshavsky. Virtual individual servers as privacy-preserving proxies for mobile devices. In *Proc. of MobiHeld*, pages 37–42. ACM, 2009.
6. H. Chen, P. Chou, S. Duri, H. Lei, and J. Reason. The design and implementation of a smart building control system. In *Proc. of International Conference on e-Business Engineering*, pages 255–262. IEEE, 2009.
7. H. Choi, S. Chakraborty, Z. M. Charbiwala, and M. B. Srivastava. Sensorsafe: a framework for privacy-preserving management of personal sensory information. In *Secure Data Management*, pages 85–100. Springer, 2011.
8. X. Chu, B. Durnota, R. Buyya, et al. Open sensor web architecture: Core services. In *Proc. of International Conference on Intelligent Sensing and Information Processing*, pages 98–103. IEEE, 2006.
9. S. Dawson-Haggerty, A. Krioukov, J. Taneja, S. Karandikar, G. Fierro, N. Kitaev, and D. Culler. Boss: Building operating system services. In *Proc. of NSDI*, 2013.
10. C. Dixon, R. Mahajan, S. Agarwal, A. Brush, B. Lee, S. Saroiu, and V. Bahl. An operating system for the home. In *Proc. of NSDI*. ACM, 2012.
11. M. Evans, B. Shui, and S. Somasundaram. Country report on building energy codes in india. In *Pacific Northwest National Laboratory, PNNL-17925*, April 2009.
12. W. Grosky, A. Kansal, S. Nath, J. Liu, and F. Zhao. Senseweb: An infrastructure for shared sensing. *Multimedia, IEEE*, 14(4):8–13, 2007.
13. X. Jiang. A High-Fidelity Energy Monitoring and Feedback Architecture for Reducing Electrical Consumption in Buildings. *PhD dissertation*, UC Berkeley 2010.
14. M. Mun, S. Hao, N. Mishra, K. Shilton, J. Burke, D. Estrin, M. Hansen, and R. Govindan. Personal data vaults: a locus of control for personal data streams. In *Proc. of Co-NEXT*, page 17. ACM, 2010.
15. A. Rowe, M. E. Berges, G. Bhatia, E. Goldman, R. Rajkumar, J. H. Garrett, J. M. Moura, and L. Soibelman. Sensor andrew: Large-scale campus-wide sensing and actuation. *IBM Journal of Research and Development*, 55(1.2):6–1, 2011.
16. K. Shilton, J. Burke, D. Estrin, R. Govindan, M. Hansen, J. Kang, and M. Mun. Designing the personal data stream: Enabling participatory privacy in mobile personal sensing. TPRC, 2009.
17. R. P. Singh, S. Keshav, and T. Brecht. A cloud-based consumer-centric architecture for energy data analytics. In *Proc. of e-Energy*. ACM, 2013.